

EXPERIMENT -3

AIM- Create an Employee–Department–Project schema. Insert at least 30 employees across 5 departments and 8 projects. Write SQL queries demonstrating selection, projection, aggregates, GROUP BY, HAVING, CASE expressions, and ORDER BY.

OBJECTIVES

- To create relational tables for Department, Employee, and Project.
- To establish relationships between the tables using Primary Keys and Foreign Keys.
- To insert at least 30 employees, 5 departments, and 8 projects.
- To retrieve specific records using Selection.
- To retrieve specific attributes using Projection.
- To perform calculations using aggregate functions such as COUNT(), AVG(), MAX(), MIN(), and SUM().
- To group records using GROUP BY.
- To filter grouped results using HAVING.
- To categorize employee data using a CASE expression.
- To sort query results using ORDER BY.

THEORY

DATABASE SCHEMA

A database schema defines the logical structure of a database, including its tables, attributes, and relationships. In this experiment, the database consists of the Department, Employee, Project, and Employee_Project relations. The Department relation stores department information, the Employee relation stores employee details, the Project relation stores project information, and the Employee_Project relation associates employees with projects. The database contains 5 departments, 8 projects, and 30 employees.

SELECTION

Selection is used to retrieve records from a relation that satisfy a specified condition. In SQL, selection is generally performed using the WHERE clause. It filters the rows of a table according to the given condition.

PROJECTION

Projection is used to retrieve only the required attributes or columns from a relation. In SQL, projection is performed using the SELECT statement by specifying the required columns. It allows only the relevant attributes to be displayed in the result.

AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a set of rows and return a single result. The aggregate functions used in this experiment are:

- COUNT() – Counts the number of rows.
- SUM() – Calculates the total of numeric values.
- AVG() – Calculates the average of numeric values.
- MAX() – Returns the maximum value.
- MIN() – Returns the minimum value.

GROUP BY

The GROUP BY clause is used to arrange rows into groups based on one or more columns. It is commonly used with aggregate functions to perform calculations separately for each group. In this experiment, it is used for department-wise analysis of employees and salaries.

HAVING

The HAVING clause is used to apply conditions to groups created using the GROUP BY clause. It is generally used with aggregate functions to filter grouped results based on a specified condition.

CASE EXPRESSION

The CASE expression is used to perform conditional operations in SQL. It evaluates specified conditions and returns a corresponding value when a condition is satisfied. In this experiment, it is used to categorize employees based on their salary.

ORDER BY

The ORDER BY clause is used to sort the records returned by a query. The results can be arranged in ascending order using ASC or descending order using DESC. In this experiment, it is used to arrange employees according to their salary.

DATABASE RELATIONSHIPS

The database represents the relationship between employees, departments, and projects. A department can have multiple employees and projects. An employee can work on multiple projects, and a project can have multiple employees. The Employee_Project relation is used to represent the association between employees and projects.

SOURCE CODE-

The screenshot displays a MySQL IDE interface with two tabs. The top tab, 'queries.sql', contains SQL code for creating a database and three tables: Department, Employee, and Project. The bottom tab, 'queries.sql', contains SQL code for inserting data into the Department and Employee tables. The interface includes a search bar, a 'Run' button, and a 'Feedback' button. The user's name, RIDDHIMA VERMA, is visible in the top right corner.

```
1 CREATE DATABASE CompanyDB;
2 USE CompanyDB;
3
4 CREATE TABLE Department (
5     DeptID INT PRIMARY KEY,
6     DeptName VARCHAR(50) NOT NULL,
7     Location VARCHAR(50)
8 );
9
10 CREATE TABLE Employee (
11     EmpID INT PRIMARY KEY,
12     EmpName VARCHAR(50) NOT NULL,
13     Salary DECIMAL(10,2),
14     JobTitle VARCHAR(50),
15     DeptID INT,
16     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
17 );
18
19 CREATE TABLE Project (
20     ProjectID INT PRIMARY KEY,
21     ProjectName VARCHAR(100) NOT NULL,
22     Budget DECIMAL(12,2),
23     DeptID INT,
24     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
25 );
26
27 TRUNCATE INTO Department VALUES
28
29
30
31
32
33
34 INSERT INTO Employee VALUES
35 (101, 'Aarav', 55000, 'Software Engineer', 1),
36 (102, 'Riya', 62000, 'Software Engineer', 1),
37 (103, 'Karan', 48000, 'System Analyst', 1),
38 (104, 'Ananya', 75000, 'Project Manager', 1),
39 (105, 'Rahul', 52000, 'Developer', 1),
40 (106, 'Priya', 68000, 'Database Admin', 1),
41 (107, 'Aditya', 58000, 'Software Engineer', 2),
42 (108, 'Sneha', 61000, 'Network Engineer', 2),
43 (109, 'Vikas', 47000, 'Support Engineer', 2),
44 (110, 'Neha', 72000, 'IT Manager', 2),
45 (111, 'Rohan', 53000, 'Web Developer', 2),
46 (112, 'Ishita', 65000, 'Cloud Engineer', 2),
47 (113, 'Aman', 45000, 'HR Executive', 3),
48 (114, 'Pooja', 50000, 'HR Executive', 3),
49 (115, 'Arjun', 68000, 'HR Manager', 3),
50 (116, 'Simran', 42000, 'Recruiter', 3),
51 (117, 'Nikhil', 47000, 'HR Executive', 3),
52 (118, 'Meera', 55000, 'Training Manager', 3),
53 (119, 'Dev', 60000, 'Accountant', 4),
```


queries.sql x + MySQL Run I/O AI Agent

```
193 GROUP BY DeptID
194 HAVING COUNT(*) >= 2;
195
196 SELECT
197     e.EmpName,
198     d.DeptName,
199     p.ProjectName
200 FROM Employee e
201 JOIN Department d
202 ON e.DeptID = d.DeptID
203 JOIN Project p
204 ON d.DeptID = p.DeptID;
205
206 SELECT
207     e.EmpName,
208     d.DeptName,
209     e.Salary,
210     CASE
211         WHEN e.Salary >= 70000 THEN 'High'
212         WHEN e.Salary >= 50000 THEN 'Medium'
213         ELSE 'Low'
214     END AS SalaryCategory
215 FROM Employee e
216 JOIN Department d
217 ON e.DeptID = d.DeptID
218 WHERE e.Salary >= 45000
219 ORDER BY e.Salary DESC;
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

OUTPUT

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

EmpID	EmpName	Salary	JobTitle	DeptID
102	Riya	62000.00	Software Engineer	1
104	Ananya	75000.00	Project Manager	1
106	Priya	68000.00	Database Admin	1
108	Sneha	61000.00	Network Engineer	2
110	Neha	72000.00	IT Manager	2
112	Ishita	65000.00	Cloud Engineer	2
115	Arjun	68000.00	HR Manager	3
120	Tanya	72000.00	Financial Analyst	4
121	Varun	85000.00	Finance Manager	4
124	Sakshi	63000.00	Financial Analyst	4
127	Kabir	70000.00	Marketing Manager	5
130	Ira	64000.00	Marketing Analyst	5

Light

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

EmpName	Salary
Aarav	55000.00
Riya	62000.00
Karan	48000.00
Ananya	75000.00
Rahul	52000.00
Priya	68000.00
Aditya	58000.00
Sneha	61000.00
Vikas	47000.00
Neha	72000.00
Rohan	53000.00
Ishita	65000.00
Aman	45000.00

Light

New History Save

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

EmpName	JobTitle	Salary
Aman		45000.00
Pooja		50000.00
Arjun		68000.00
Simran		42000.00
Nikhil		47000.00
Meera		55000.00
Dev		60000.00
Tanya		72000.00
Varun		85000.00
Kriti		54000.00
Mohit		49000.00
Sakshi		63000.00
Yash		50000.00
Naina		58000.00
Kabir		70000.00
Diya		46000.00
Aryan		52000.00
Ira		64000.00

Light

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

EmpName	JobTitle
Aarav	Software Engineer
Riya	Software Engineer
Karan	System Analyst
Ananya	Project Manager
Rahul	Developer
Priya	Database Admin
Aditya	Software Engineer
Sneha	Network Engineer
Vikas	Support Engineer
Neha	IT Manager
Rohan	Web Developer
Ishita	Cloud Engineer
Aman	HR Executive
Pooja	HR Executive
Arjun	HR Manager
Simran	Recruiter
Nikhil	HR Executive
Meera	Training Manager

Light

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

EmpName	JobTitle
Dev	Accountant
Tanya	Financial Analyst
Varun	Finance Manager
Kriti	Accountant
Mohit	Auditor
Sakshi	Financial Analyst
Yash	Marketing Executive
Naina	Marketing Executive
Kabir	Marketing Manager
Diya	Content Writer
Aryan	SEO Specialist
Ira	Marketing Analyst

DeptID	EmployeeCount
1	6
2	6
3	6
4	6
5	6

DeptID	AverageSalary
1	60000.000000
2	59333.333333
3	51166.666667
4	63833.333333
5	56666.666667

DeptID	TotalSalary
1	360000.00

Light

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

MinimumSalary
42000.00

AverageSalary
58200.000000

TotalSalary
1746000.00

DeptID	EmployeeCount
1	6
2	6
3	6
4	6
5	6

Light

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

DeptID	EmployeeCount
1	6
2	6
3	6
4	6
5	6

DeptID	AverageSalary
1	60000.000000
2	59333.333333
3	51166.666667
4	63833.333333
5	56666.666667

DeptID	TotalSalary
1	360000.00

Light

RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

DeptID	TotalSalary
1	360000.00
2	356000.00
3	307000.00
4	383000.00
5	340000.00

DeptName	EmployeeCount	AverageSalary
CSE	6	60000.000000
IT	6	59333.333333
HR	6	51166.666667
Finance	6	63833.333333
Marketing	6	56666.666667

DeptID	EmployeeCount
1	6

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

DeptID	EmployeeCount
1	6
2	6
3	6
4	6
5	6

DeptID	AverageSalary
4	63833.333333

EmpName	Salary	SalaryCategory
Aarav	55000.00	Medium Salary
Riya	62000.00	Medium Salary
Karan	48000.00	Low Salary
Ananya	75000.00	High Salary

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

Karan	48000.00	Low Salary
Ananya	75000.00	High Salary
Rahul	52000.00	Medium Salary
Priya	68000.00	Medium Salary
Aditya	58000.00	Medium Salary
Sneha	61000.00	Medium Salary
Vikas	47000.00	Low Salary
Neha	72000.00	High Salary
Rohan	53000.00	Medium Salary
Ishita	65000.00	Medium Salary
Aman	45000.00	Low Salary
Pooja	50000.00	Medium Salary
Arjun	68000.00	Medium Salary
Simran	42000.00	Low Salary
Nikhil	47000.00	Low Salary
Meera	55000.00	Medium Salary
Dev	60000.00	Medium Salary
Tanya	72000.00	High Salary
Varun	85000.00	High Salary
Kriti	54000.00	Medium Salary
Mohit	49000.00	Low Salary

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

Mohit	49000.00	Low Salary
Sakshi	63000.00	Medium Salary
Yash	50000.00	Medium Salary
Naina	58000.00	Medium Salary
Kabir	70000.00	High Salary
Diya	46000.00	Low Salary
Aryan	52000.00	Medium Salary
Ira	64000.00	Medium Salary

EmpName	JobTitle	Salary	EmployeeLevel
Aarav	Software Engineer	55000.00	Mid Level
Riya	Software Engineer	62000.00	Mid Level
Karan	System Analyst	48000.00	Junior Level
Ananya	Project Manager	75000.00	Senior Level
Rahul	Developer	52000.00	Mid Level
Priya	Database Admin	68000.00	Mid Level
Aditya	Software Engineer	58000.00	Mid Level
Sneha	Network Engineer	61000.00	Mid Level
Vikas	Support Engineer	47000.00	Junior Level

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

Mohit	Auditor	49000.00	Junior Level
Sakshi	Financial Analyst	63000.00	Mid Level
Yash	Marketing Executive	50000.00	Mid Level
Naina	Marketing Executive	58000.00	Mid Level
Kabir	Marketing Manager	70000.00	Senior Level
Diya	Content Writer	46000.00	Junior Level
Aryan	SEO Specialist	52000.00	Mid Level
Ira	Marketing Analyst	64000.00	Mid Level

EmpName	Salary
Varun	85000.00
Ananya	75000.00
Tanya	72000.00
Neha	72000.00
Kabir	70000.00
Priya	68000.00
Arjun	68000.00
Ishita	65000.00
Ira	64000.00

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

Ira	64000.00
Sakshi	63000.00
Riya	62000.00
Sneha	61000.00
Dev	60000.00
Naina	58000.00
Aditya	58000.00
Aarav	55000.00
Meera	55000.00
Kriti	54000.00
Rohan	53000.00
Rahul	52000.00
Aryan	52000.00
Pooja	50000.00
Yash	50000.00
Mohit	49000.00
Karan	48000.00
Nikhil	47000.00
Vikas	47000.00
Diya	46000.00
Aman	45000.00

Light

RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

Simran	42000.00
--------	----------

EmpName	Salary
Aarav	55000.00
Aditya	58000.00
Aman	45000.00
Ananya	75000.00
Arjun	68000.00
Aryan	52000.00
Dev	60000.00
Diya	46000.00
Ira	64000.00
Ishita	65000.00
Kabir	70000.00
Karan	48000.00
Kriti	54000.00
Meera	55000.00
Mohit	49000.00
Naina	58000.00

Light

101 ms

STDIN

Input for the program (Optional)

Output:

Nazma	58000.00
Neha	72000.00
Nikhil	47000.00
Pooja	50000.00
Priya	68000.00
Rahul	52000.00
Riya	62000.00
Rohan	53000.00
Sakshi	63000.00
Simran	42000.00
Sneha	61000.00
Tanya	72000.00
Varun	85000.00
Vikas	47000.00
Yash	50000.00

EmpName	DeptID	Salary
Ananya	1	75000.00
Priya	1	68000.00

101 ms

STDIN

Input for the program (Optional)

Output:

Riya	1	62000.00
Aarav	1	55000.00
Rahul	1	52000.00
Karan	1	48000.00
Neha	2	72000.00
Ishita	2	65000.00
Sneha	2	61000.00
Aditya	2	58000.00
Rohan	2	53000.00
Vikas	2	47000.00
Arjun	3	68000.00
Meera	3	55000.00
Pooja	3	50000.00
Nikhil	3	47000.00
Anan	3	45000.00
Simran	3	42000.00
Varun	4	85000.00
Tanya	4	72000.00
Sakshi	4	63000.00
Dev	4	60000.00

101 ms

STDIN

Input for the program (Optional)

Output:

Kriti	4	54000.00
Mohit	4	49000.00
Kabir	5	70000.00
Ira	5	64000.00
Naina	5	58000.00
Aryan	5	52000.00
Yash	5	50000.00
Diya	5	46000.00

ProjectID	ProjectName	Budget	DeptID
201	E-Commerce Website	500000.00	1
202	AI Chatbot	750000.00	1
203	Cloud Migration	600000.00	2
204	Network Security	450000.00	2
205	Employee Training	200000.00	3
206	Payroll System	350000.00	4
207	Digital Marketing	300000.00	5
208	Market Research	250000.00	5



RIDDHIMA VERMA

I/O

AI Agent

101 ms

STDIN

Input for the program (Optional)

Output:

ProjectName	Budget	
E-Commerce Website	500000.00	
AI Chatbot	750000.00	
Cloud Migration	600000.00	
Network Security	450000.00	
TotalProjectBudget	3400000.00	
AverageProjectBudget	425000.000000	
DeptID	NumberOfProjects	TotalBudget

 I/O

 AI Agent

 RIDDHIMA VERMA

101 ms 

STDIN

Input for the program (Optional)

Output:

DeptID	NumberOfProjects	TotalBudget
1	2	1250000.00
2	2	1050000.00
3	1	200000.00
4	1	350000.00
5	2	550000.00

DeptID	NumberOfProjects
1	2
2	2
5	2

EmpName	DeptName	ProjectName
Aarav	CSE	E-Commerce Website
Riya	CSE	E-Commerce Website

light

101 ms

STDIN

Input for the program (Optional)

Output:

Karan	CSE	E-Commerce Website
Ananya	CSE	E-Commerce Website
Rahul	CSE	E-Commerce Website
Priya	CSE	E-Commerce Website
Aarav	CSE	AI Chatbot
Riya	CSE	AI Chatbot
Karan	CSE	AI Chatbot
Ananya	CSE	AI Chatbot
Rahul	CSE	AI Chatbot
Priya	CSE	AI Chatbot
Aditya	IT	Cloud Migration
Sneha	IT	Cloud Migration
Vikas	IT	Cloud Migration
Neha	IT	Cloud Migration
Rohan	IT	Cloud Migration
Ishita	IT	Cloud Migration
Aditya	IT	Network Security
Sneha	IT	Network Security
Vikas	IT	Network Security
Neha	IT	Network Security
Rohan	IT	Network Security

 RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

Ishita	IT	Network Security	
Aman	HR	Employee Training	
Pooja	HR	Employee Training	
Arjun	HR	Employee Training	
Simran	HR	Employee Training	
Nikhil	HR	Employee Training	
Meera	HR	Employee Training	
Dev	Finance	Payroll System	
Tanya	Finance	Payroll System	
Varun	Finance	Payroll System	
Kriti	Finance	Payroll System	
Mohit	Finance	Payroll System	
Sakshi	Finance	Payroll System	
Yash	Marketing	Digital Marketing	
Naina	Marketing	Digital Marketing	
Kabir	Marketing	Digital Marketing	
Diya	Marketing	Digital Marketing	
Aryan	Marketing	Digital Marketing	
Ira	Marketing	Digital Marketing	
Yash	Marketing	Market Research	
Naina	Marketing	Market Research	

Light

 RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

Kabir	Marketing	Market Research	
Diya	Marketing	Market Research	
Aryan	Marketing	Market Research	
Ira	Marketing	Market Research	

+-----+

EmpName	DeptName	Salary	SalaryCategory
Varun	Finance	85000.00	High
Ananya	CSE	75000.00	High
Tanya	Finance	72000.00	High
Neha	IT	72000.00	High
Kabir	Marketing	70000.00	High
Priya	CSE	68000.00	Medium
Arjun	HR	68000.00	Medium
Ishita	IT	65000.00	Medium
Ira	Marketing	64000.00	Medium
Sakshi	Finance	63000.00	Medium
Riya	CSE	62000.00	Medium
Sneha	IT	61000.00	Medium
Dev	Finance	60000.00	Medium

Light

 RIDDHIMA VERMA

I/O AI Agent 101 ms

STDIN

Input for the program (Optional)

Output:

Riya	CSE	62000.00	Medium
Sneha	IT	61000.00	Medium
Dev	Finance	60000.00	Medium
Naina	Marketing	58000.00	Medium
Aditya	IT	58000.00	Medium
Aarav	CSE	55000.00	Medium
Meera	HR	55000.00	Medium
Kriti	Finance	54000.00	Medium
Rohan	IT	53000.00	Medium
Rahul	CSE	52000.00	Medium
Aryan	Marketing	52000.00	Medium
Yash	Marketing	50000.00	Medium
Pooja	HR	50000.00	Medium
Mohit	Finance	49000.00	Low
Karan	CSE	48000.00	Low
Nikhil	HR	47000.00	Low
Vikas	IT	47000.00	Low
Diya	Marketing	46000.00	Low
Aman	HR	45000.00	Low

Light

OBSERVATION/OUTCOME

1. Employee, Department, and Project tables were successfully created.
2. Data for 30 employees, 5 departments, and 8 projects was successfully inserted.
3. Selection and projection queries successfully retrieved the required records and attributes.
4. Aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN() were successfully executed.
5. GROUP BY and HAVING were used to group and filter grouped data.
6. CASE expression successfully categorized employees based on salary.
7. ORDER BY successfully sorted employee records according to the specified conditions.
8. Queries using joins successfully retrieved related data from multiple tables.

EXPERIMENT -4

AIM- Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

OBJECTIVES

1. To understand and implement different types of SQL joins.
2. To retrieve related data using INNER JOIN and LEFT JOIN.
3. To understand and implement a self-join on the Employee table.
4. To perform a 3-way join between Employee, Department, and Project tables.
5. To understand and implement correlated subqueries.
6. To use the EXISTS operator for checking the existence of related records.
7. To simulate the functionality of INTERSECT and EXCEPT using SQL queries.
8. To analyze and compare the execution plans of SQL queries using EXPLAIN.

THEORY

INNER JOIN

An INNER JOIN returns only those records that have matching values in both tables based on the specified join condition. It is used to combine related information from two or more tables.

For example, the Employee and Department tables can be joined using the common DeptID attribute to display employees along with their departments.

LEFT JOIN

A LEFT JOIN returns all records from the left table and the matching records from the right table. If no matching record exists in the right table, NULL values are returned for the columns of the right table.

It is useful when all records from the primary or left-side table must be retained, even if they do not have corresponding records in the other table.

SELF-JOIN

A self-join is a join in which a table is joined with itself. Different aliases are assigned to the same table so that its records can be compared with one another.

It can be used to find relationships between employees, such as employees working in the same department or employees having similar salaries.

3-WAY JOIN

A 3-way join combines three tables in a single query using related attributes.

In the Employee schema, the Employee table can be joined with both Department and Project tables. This allows information such as employee name, department name, and project name to be retrieved together.

CORRELATED SUBQUERY

A correlated subquery is a subquery that depends on a value from the outer query. Unlike an independent subquery, it is logically evaluated for each row considered by the outer query.

Correlated subqueries are useful when the result of the inner query needs to be compared with the current row of the outer query, such as finding employees whose salary is greater than the average salary of their own department.

EXISTS

The EXISTS operator checks whether a subquery returns at least one row.

It returns TRUE when the subquery produces one or more matching records and FALSE when no matching record exists. It is commonly used to determine whether a related record exists in another table.

SIMULATED INTERSECT-

INTERSECT returns records that are common to the results of two queries.

When INTERSECT is not supported by a particular SQL system, its functionality can be simulated using alternatives such as INNER JOIN or EXISTS. The columns and data types involved in the comparison should be compatible.

SIMULATED EXCEPT-

EXCEPT returns records that are present in the first query result but absent from the second query result.

When EXCEPT is not supported, it can be simulated using NOT EXISTS or an appropriate LEFT JOIN with a NULL check

EXPLAIN

EXPLAIN is used to display the execution plan that the database optimizer intends to use for a SQL query.

It provides information about how tables will be accessed and joined, such as:

- Table scans
- Index usage
- Join methods
- Estimated number of rows
- Filtering operations
- Query execution cost

Comparing execution plans helps in understanding query performance and identifying inefficient queries. An execution plan with fewer scanned rows or an appropriate index access path may indicate a more efficient execution strategy, although the exact interpretation depends on the database system and data distribution.

SOURCE CODE

```
queries.sql x + MySQL Run I/O AI Agent RIDDHIMA VERMA

1 CREATE DATABASE CompanyDB;
2 USE CompanyDB;
3
4 CREATE TABLE Department (
5     DeptID INT PRIMARY KEY,
6     DeptName VARCHAR(50),
7     Location VARCHAR(50)
8 );
9
10 CREATE TABLE Employee (
11     EmpID INT PRIMARY KEY,
12     EmpName VARCHAR(50),
13     Salary DECIMAL(10,2),
14     JobTitle VARCHAR(50),
15     DeptID INT,
16     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
17 );
18
19 CREATE TABLE Project (
20     ProjectID INT PRIMARY KEY,
21     ProjectName VARCHAR(100),
22     Budget DECIMAL(12,2),
23     DeptID INT,
24     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
25 );
26
27 INSERT INTO Department VALUES
```

STDIN
Input for the program (optional)

Output:
Click on RUN button to see the output

Ready Feedback New History Save

```
queries.sql x + MySQL Run I/O AI Agent RIDDHIMA VERMA

27 INSERT INTO Department VALUES
28 (1, 'CSE', 'Delhi'),
29 (2, 'IT', 'Mumbai'),
30 (3, 'HR', 'Bangalore'),
31 (4, 'Finance', 'Pune'),
32 (5, 'Marketing', 'Hyderabad');
33
34 INSERT INTO Employee VALUES
35 (101, 'Aarav', 55000, 'Software Engineer', 1),
36 (102, 'Riya', 62000, 'Software Engineer', 1),
37 (103, 'Karan', 48000, 'System Analyst', 1),
38 (104, 'Ananya', 75000, 'Project Manager', 1),
39 (105, 'Rahul', 52000, 'Developer', 1),
40 (106, 'Priya', 68000, 'Database Admin', 1),
41 (107, 'Aditya', 58000, 'Software Engineer', 2),
42 (108, 'Sneha', 61000, 'Network Engineer', 2),
43 (109, 'Vikas', 47000, 'Support Engineer', 2),
44 (110, 'Neha', 72000, 'IT Manager', 2),
45 (111, 'Rohan', 53000, 'Web Developer', 2),
46 (112, 'Ishita', 65000, 'Cloud Engineer', 2),
47 (113, 'Aman', 45000, 'HR Executive', 3),
48 (114, 'Pooja', 50000, 'HR Executive', 3),
49 (115, 'Arjun', 68000, 'HR Manager', 3),
50 (116, 'Simran', 42000, 'Recruiter', 3),
51 (117, 'Nikhil', 47000, 'HR Executive', 3),
52 (118, 'Meera', 55000, 'Training Manager', 3),
53 (119, 'Dev', 60000, 'Accountant', 4);
```

STDIN
Input for the program (optional)

Output:
Click on RUN button to see the output

Ready Feedback New History Save

```
queries.sql x + MySQL Run I/O AI Agent RIDDHIMA VERMA

54 (120, 'Tanya', 72000, 'Financial Analyst', 4),
55 (121, 'Varun', 85000, 'Finance Manager', 4),
56 (122, 'Kriti', 54000, 'Accountant', 4),
57 (123, 'Mohit', 49000, 'Auditor', 4),
58 (124, 'Sakshi', 63000, 'Financial Analyst', 4),
59 (125, 'Yash', 50000, 'Marketing Executive', 5),
60 (126, 'Naina', 58000, 'Marketing Executive', 5),
61 (127, 'Kabin', 70000, 'Marketing Manager', 5),
62 (128, 'Diya', 46000, 'Content Writer', 5),
63 (129, 'Aryan', 52000, 'SEO Specialist', 5),
64 (130, 'Ira', 64000, 'Marketing Analyst', 5);
65
66 INSERT INTO Project VALUES
67 (201, 'E-Commerce Website', 500000, 1),
68 (202, 'AI Chatbot', 750000, 1),
69 (203, 'Cloud Migration', 600000, 2),
70 (204, 'Network Security', 450000, 2),
71 (205, 'Employee Training', 200000, 3),
72 (206, 'Payroll system', 350000, 4),
73 (207, 'Digital Marketing', 300000, 5),
74 (208, 'Market Research', 250000, 5);
75
76 SELECT
77     e.EmpName,
78     d.DeptName
79 FROM Employee e
80 INNER JOIN Department d
```

STDIN
Input for the program (optional)

Output:
Click on RUN button to see the output

Ready Feedback New History Save

OUTPUT

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

EmpName	DeptName
Aarav	CSE
Riya	CSE
Karan	CSE
Ananya	CSE
Rahul	CSE
Priya	CSE
Aditya	IT
Sneha	IT
Vikas	IT
Neha	IT
Rohan	IT
Ishita	IT
Aman	HR
Pooja	HR
Arjun	HR
Simran	HR
Nikhil	HR

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

DeptName	EmpName
CSE	Aarav
CSE	Riya
CSE	Karan

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

CSE	Ananya
CSE	Rahul
CSE	Priya
IT	Aditya
IT	Sneha
IT	Vikas
IT	Neha
IT	Rohan
IT	Ishita
HR	Aman
HR	Pooja
HR	Arjun
HR	Simran
HR	Nikhil
HR	Meera
Finance	Dev
Finance	Tanya
Finance	Varun
Finance	Kriti
Finance	Mohit

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Finance	Sakshi
Marketing	Yash
Marketing	Naina
Marketing	Kabir
Marketing	Diya
Marketing	Aryan
Marketing	Ira

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Employee1	Employee2	DeptID
Aarav	Riya	1
Aarav	Karan	1
Aarav	Ananya	1
Aarav	Rahul	1
Aarav	Priya	1
Riya	Karan	1
Riya	Ananya	1
Riya	Rahul	1
Riya	Priya	1

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Karan	Ananya	
Karan	Rahul	1
Karan	Priya	1
Ananya	Rahul	1
Ananya	Priya	1
Rahul	Priya	1
Aditya	Sneha	2
Aditya	Vikas	2
Aditya	Neha	2
Aditya	Rohan	2
Aditya	Ishita	2
Sneha	Vikas	2
Sneha	Neha	2
Sneha	Rohan	2
Sneha	Ishita	2
Vikas	Neha	2
Vikas	Rohan	2
Vikas	Ishita	2
Neha	Rohan	2
Neha	Ishita	2

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Aman	Pooja	
Aman	Arjun	3
Aman	Simran	3
Aman	Nikhil	3
Aman	Meera	3
Pooja	Arjun	3
Pooja	Simran	3
Pooja	Nikhil	3
Pooja	Meera	3
Arjun	Simran	3
Arjun	Nikhil	3
Arjun	Meera	3
Simran	Nikhil	3
Simran	Meera	3
Nikhil	Meera	3
Dev	Tanya	4
Dev	Varun	4
Dev	Kriti	4
Dev	Mohit	4
Dev	Sakshi	4

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

Tanya	Varun	4
Tanya	Kriti	4
Tanya	Mohit	4
Tanya	Sakshi	4
Varun	Kriti	4
Varun	Mohit	4
Varun	Sakshi	4
Kriti	Mohit	4
Kriti	Sakshi	4
Mohit	Sakshi	4
Yash	Naina	5
Yash	Kabir	5
Yash	Diya	5
Yash	Aryan	5
Yash	Ira	5
Naina	Kabir	5
Naina	Diya	5
Naina	Aryan	5
Naina	Ira	5
Kabir	Diya	5

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

Kabir	Aryan	5
Kabir	Ira	5
Diya	Aryan	5
Diya	Ira	5
Aryan	Ira	5

EmpName	DeptName	ProjectName
Aarav	CSE	E-Commerce Website
Riya	CSE	E-Commerce Website
Karan	CSE	E-Commerce Website
Ananya	CSE	E-Commerce Website
Rahul	CSE	E-Commerce Website
Priya	CSE	E-Commerce Website
Aarav	CSE	AI Chatbot
Riya	CSE	AI Chatbot
Karan	CSE	AI Chatbot
Ananya	CSE	AI Chatbot
Rahul	CSE	AI Chatbot

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

Priya	CSE	AI Chatbot
Aditya	IT	Cloud Migration
Sneha	IT	Cloud Migration
Vikas	IT	Cloud Migration
Neha	IT	Cloud Migration
Rohan	IT	Cloud Migration
Ishita	IT	Cloud Migration
Aditya	IT	Network Security
Sneha	IT	Network Security
Vikas	IT	Network Security
Neha	IT	Network Security
Rohan	IT	Network Security
Ishita	IT	Network Security
Aman	HR	Employee Training
Pooja	HR	Employee Training
Arjun	HR	Employee Training
Simran	HR	Employee Training
Nikhil	HR	Employee Training
Meera	HR	Employee Training
Dev	Finance	Payroll System

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

Tanya	Finance	Payroll System
Varun	Finance	Payroll System
Kriti	Finance	Payroll System
Mohit	Finance	Payroll System
Sakshi	Finance	Payroll System
Yash	Marketing	Digital Marketing
Naina	Marketing	Digital Marketing
Kabir	Marketing	Digital Marketing
Diya	Marketing	Digital Marketing
Aryan	Marketing	Digital Marketing
Ira	Marketing	Digital Marketing
Yash	Marketing	Market Research
Naina	Marketing	Market Research
Kabir	Marketing	Market Research
Diya	Marketing	Market Research
Aryan	Marketing	Market Research
Ira	Marketing	Market Research

EmpName	Salary	DeptID
---------	--------	--------

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

EmpName	Salary	DeptID
Riya	62000.00	1
Ananya	75000.00	1
Priya	68000.00	1
Sneha	61000.00	2
Neha	72000.00	2
Ishita	65000.00	2
Arjun	68000.00	3
Meera	55000.00	3
Tanya	72000.00	4
Varun	85000.00	4
Naina	58000.00	5
Kabir	70000.00	5
Ira	64000.00	5

DeptName
CSE

I/O
AI Agent
92 ms

STDIN

Input for the program (Optional)

Output:

IT
HR
Finance
Marketing

EmpName
Aarav
Riya
Karan
Ananya
Rahul
Priya
Aditya
Sneha
Vikas
Neha
Rohan
Ishita

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Aman
Pooja
Arjun
Simran
Nikhil
Meera
Dev
Tanya
Varun
Kriti
Mohit
Sakshi
Yash
Naina
Kabir
Diya
Aryan
Ira

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

EmpName
Aarav
Aarav
Riya
Riya
Karan
Karan
Ananya
Ananya
Rahul
Rahul
Priya
Priya
Aditya
Aditya
Sneha
Sneha
Vikas

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Vikas
Neha
Neha
Rohan
Rohan
Ishita
Ishita
Aman
Pooja
Arjun
Simran
Nikhil
Meera
Dev
Tanya
Varun
Kriti
Mohit
Sakshi
Yash

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Yash
Naina
Naina
Kabir
Kabir
Diya
Diya
Aryan
Aryan
Ira
Ira
EmpName
Aarav
Riya
Karan
Ananya
Rahul

RIDDHIMA VERMA

I/O
AI Agent
92 ms

STDIN
Input for the program (Optional)

Output:

Priya
Aditya
Sneha
Vikas
Neha
Rohan
Ishita
Aman
Pooja
Arjun
Simran
Nikhil
Meera
Dev
Tanya
Varun
Kriti
Mohit
Sakshi
Yash

OBSERVATIONS/OUTCOME

1. Employee, Department, and Project tables were successfully created and populated with the required data.
2. INNER JOIN successfully retrieved matching employee and department records.
3. LEFT JOIN successfully displayed all departments along with their corresponding employees.
4. A self-join successfully identified employees belonging to the same department.
5. A 3-way join successfully retrieved employee, department, and project details together.
6. The correlated subquery successfully identified employees whose salary was greater than the average salary of their respective department.
7. The EXISTS operator successfully checked for the existence of projects associated with employees' departments.
8. INTERSECT and EXCEPT operations were successfully simulated using EXISTS and NOT EXISTS.
9. EXPLAIN successfully displayed the execution plans for different SQL queries, allowing their execution strategies to be examined and compared.

EXPERIMENT -5

AIM- Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

OBJECTIVES

1. To understand the concept and purpose of SQL views.
2. To create a view for displaying department-wise salary summaries.
3. To create a view representing the employee-manager hierarchy.
4. To test whether the created views are updatable.
5. To understand the limitations of updating views containing aggregate functions and joins.
6. To understand Common Table Expressions (CTEs).
7. To implement a recursive CTE for traversing employee-manager relationships.
8. To display complete reporting chains from employees to the top-level manager.

THEORY

SQL View

A **view** is a virtual table created from the result of a SQL query. It does not normally store the actual data separately; instead, it stores the query definition used to retrieve the data from underlying tables.

Views are useful for:

- Simplifying complex queries.
- Providing a customized representation of data.
- Restricting access to specific columns or rows.
- Improving data abstraction.
- Reusing frequently required queries.

Department Salary Summary View

A department salary summary view combines employee and department information and uses aggregate functions such as COUNT() and AVG() to provide summarized salary information for each department.

For example, it can display:

- Department name
- Number of employees
- Total salary
- Average salary
- Maximum salary
- Minimum salary

Because this view contains **aggregate functions and GROUP BY**, it is generally not directly updatable.

Employee Hierarchy View

An employee hierarchy represents the relationship between an employee and their manager.

For this purpose, the Employee table contains a ManagerID attribute that refers back to EmpID in the same table.

Updatable View

An updatable view is a view through which operations such as INSERT, UPDATE, or DELETE can be performed on the underlying table.

A simple view based on a single table is generally more likely to be updatable.

Non-Updatable Views

A view may not be directly updatable when its definition contains features that make it difficult to map the result back to individual base-table rows.

Common examples include views containing:

- Aggregate functions
- GROUP BY
- DISTINCT
- Set operations
- Certain joins
- Calculated or derived results

Therefore, a department salary summary containing SUM(), AVG(), and GROUP BY cannot normally be directly updated.

Similarly, the employee hierarchy view uses a self-join, so it should not be treated as a simple updatable view.

Common Table Expression

A **Common Table Expression (CTE)** is a temporary named result set that can be referenced within a SQL statement.

It is defined using the WITH clause.

Recursive CTE

A **recursive CTE** is a CTE that refers to itself. It is useful for processing hierarchical or tree-structured data.

A recursive CTE normally consists of two parts:

1. **Anchor member**: identifies the starting records.
2. **Recursive member**: repeatedly finds the next level of related records.

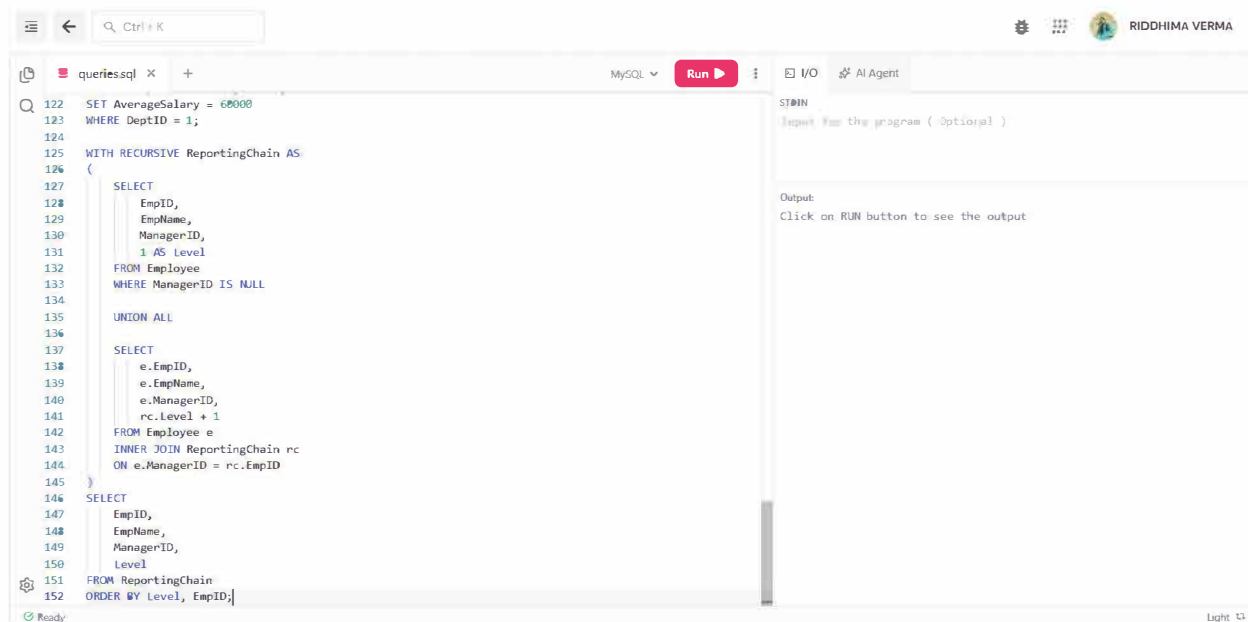
SOURCE CODE

The image displays three sequential screenshots of a MySQL IDE interface, showing the execution of SQL queries. The user's name, RIDDHIMA VERMA, is visible in the top right corner of each window.

First Screenshot: The SQL editor shows the initial database setup. It starts with dropping the database if it exists, then creating it. Two tables are created: 'Department' with columns DeptID (Primary Key), DeptName, and Location; and 'Employee' with columns EmpID (Primary Key), EmpName, Salary, JobTitle, and DeptID (Foreign Key to Department). A third table, 'Project', is also created with columns ProjectID (Primary Key), ProjectName, Budget, and DeptID (Foreign Key to Department). The interface includes a 'Run' button and a status bar indicating 'Ready'.

Second Screenshot: This screenshot shows the insertion of data into the 'Department' table. The SQL code includes an 'INSERT INTO Department VALUES' statement with five rows of data: (1, 'CSE', 'Delhi'), (2, 'IT', 'Mumbai'), (3, 'HR', 'Bangalore'), (4, 'Finance', 'Pune'), and (5, 'Marketing', 'Hyderabad'). The 'Run' button is highlighted, and the output pane shows the message 'Click on RUN button to see the output'.

Third Screenshot: This screenshot shows the insertion of data into the 'Employee' table. The SQL code includes an 'INSERT INTO Employee VALUES' statement with 19 rows of data, including employee names, salaries, job titles, and department IDs. The 'Run' button is highlighted, and the output pane shows the message 'Click on RUN button to see the output'.



OUTPUT

RIDDHIMA VERMA

I/O
AI Agent
117 ms

STDIN

Input for the program (Optional)

Output:

DeptID	DeptName	EmployeeCount	TotalSalary	AverageSalary	MaximumSalary	MinimumSalary
1	CSE	6	360000.00	60000.000000	75000.00	48000.00
2	IT	6	356000.00	59333.333333	72000.00	47000.00
3	HR	6	307000.00	51166.666667	68000.00	42000.00
4	Finance	6	383000.00	63833.333333	85000.00	49000.00
5	Marketing	6	340000.00	56666.666667	70000.00	46000.00

EmpID	EmpName	JobTitle	Salary	ManagerID	ManagerName
101	Aarav	Software Engineer	55000.00	NULL	NULL
102	Riya	Software Engineer	62000.00	101	Aarav
103	Karan	System Analyst	48000.00	102	Riya
104	Ananya	Project Manager	75000.00	102	Riya
105	Rahul	Developer	52000.00	101	Aarav
106	Priya	Database Admin	68000.00	101	Aarav
107	Aditya	Software Engineer	58000.00	NULL	NULL
108	Sneha	Network Engineer	61000.00	107	Aditya
109	Vikas	Support Engineer	47000.00	108	Sneha
110	Neha	IT Manager	72000.00	107	Aditya
111	Rohan	Web Developer	53000.00	110	Neha
112	Ishita	Cloud Engineer	65000.00	110	Neha

RIDDHIMA VERMA

I/O
AI Agent
117 ms

STDIN

Input for the program (Optional)

Output:

113	Aman	HR Executive	45000.00	NULL	NULL
114	Pooja	HR Executive	50000.00	113	Aman
115	Arjun	HR Manager	68000.00	113	Aman
116	Simran	Recruiter	42000.00	115	Arjun
117	Nikhil	HR Executive	47000.00	115	Arjun
118	Meera	Training Manager	55000.00	113	Aman
119	Dev	Accountant	60000.00	NULL	NULL
120	Tanya	Financial Analyst	72000.00	119	Dev
121	Varun	Finance Manager	85000.00	119	Dev
122	Kriti	Accountant	54000.00	121	Varun
123	Mohit	Auditor	49000.00	121	Varun
124	Sakshi	Financial Analyst	63000.00	119	Dev
125	Yash	Marketing Executive	50000.00	NULL	NULL
126	Naina	Marketing Executive	58000.00	125	Yash
127	Kabir	Marketing Manager	70000.00	125	Yash
128	Diya	Content Writer	46000.00	127	Kabir
129	Aryan	SEO Specialist	52000.00	127	Kabir
130	Ira	Marketing Analyst	64000.00	129	Aryan

DeptID	DeptName	EmployeeCount	TotalSalary	AverageSalary	MaximumSalary	MinimumSalary
1	CSE	6	365000.00	60833.333333	75000.00	48000.00

OBSERVATIONS/OUTCOME

1. A department salary summary view was successfully created using aggregate functions and GROUP BY.
2. An employee hierarchy view was successfully created using a self-join on the Employee table.
3. A ManagerID attribute was added to establish the employee-manager relationship.
4. A simple employee salary view was successfully updated, demonstrating an updatable view.
5. The department salary summary view was found to be non-updatable because it contains aggregate functions and GROUP BY.
6. The employee hierarchy view is based on a self-join and is not treated as a directly updatable view.
7. A recursive CTE was successfully implemented to traverse the employee-manager hierarchy.
8. The recursive CTE displayed the reporting level and complete reporting chain for employees.

EXPERIMENT -6

AIM- Create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

OBJECTIVES

1. To understand the concept of stored procedures in SQL.
2. To create a stored procedure for transferring an employee between departments.
3. To implement input validation and error handling using SIGNAL.
4. To understand SQL triggers and their execution timing.
5. To create a trigger for validating employee salaries.
6. To create an audit table for maintaining salary-change history.
7. To create an audit trigger for recording salary modifications.
8. To test valid and invalid inputs using different edge cases.

THEORY

Stored Procedure

A stored procedure is a named collection of SQL statements stored in the database and executed when called.

Stored procedures are useful for:

- Reusing frequently performed operations.
- Encapsulating business logic.
- Performing validation.
- Handling transactions.

Parameterized Stored Procedure

A stored procedure can accept parameters from the user or application.

For example: `transfer_employee(emp_id, new_dept_id)`

Here:

- `emp_id` identifies the employee.
- `new_dept_id` identifies the destination department.

Error Handling

Error handling allows a procedure to respond appropriately when an invalid condition occurs.

For example, an error can be generated when:

- Employee does not exist.
- Department does not exist.
- Employee is already in the requested department.
- Invalid input is supplied.

Transaction

A transaction groups multiple database operations into a single logical unit.

The main transaction commands are:

- START TRANSACTION
- COMMIT
- ROLLBACK

Trigger

A trigger is a database object that automatically executes when a specified event occurs on a table.

Triggers can execute:

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER UPDATE
- BEFORE DELETE
- AFTER DELETE

Salary Validation Trigger

A salary validation trigger can ensure that invalid salary values are not inserted or updated.

For example, a salary can be required to be greater than zero.

A BEFORE INSERT trigger validates newly inserted employees, while a BEFORE UPDATE trigger validates salary changes.

Audit Logging

Audit logging records important changes made to database records.

For salary changes, an audit table can store:

- Employee ID
- Old salary
- New salary
- Date and time of change
- Type of operation

SOURCE CODE

```
1 DROP DATABASE IF EXISTS CompanyDB;
2 CREATE DATABASE CompanyDB;
3 USE CompanyDB;
4
5 CREATE TABLE Department (
6     DeptID INT PRIMARY KEY,
7     DeptName VARCHAR(50),
8     Location VARCHAR(50)
9 );
10
11 CREATE TABLE Employee (
12     EmpID INT PRIMARY KEY,
13     EmpName VARCHAR(50),
14     Salary DECIMAL(10,2),
15     JobTitle VARCHAR(50),
16     DeptID INT,
17     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
18 );
19
20 CREATE TABLE EmployeeAudit (
21     AuditID INT AUTO_INCREMENT PRIMARY KEY,
22     EmpID INT,
23     OldSalary DECIMAL(10,2),
24     NewSalary DECIMAL(10,2),
25     OldDeptID INT,
26     NewDeptID INT,
27     ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
28     ActionType VARCHAR(50)
29 );
30
```

```
29 INSERT INTO Department VALUES
30 (1, 'CSE', 'Delhi'),
31 (2, 'IT', 'Mumbai'),
32 (3, 'HR', 'Bangalore'),
33 (4, 'Finance', 'Pune'),
34 (5, 'Marketing', 'Hyderabad');
35
36 INSERT INTO Employee VALUES
37 (101, 'Aarav', 55000, 'Software Engineer', 1),
38 (102, 'Riya', 62000, 'Software Engineer', 1),
39 (103, 'Karan', 48000, 'System Analyst', 1),
40 (104, 'Ananya', 75000, 'Project Manager', 2),
41 (105, 'Rahul', 52000, 'Developer', 2),
42 (106, 'Priya', 68000, 'Database Admin', 3),
43 (107, 'Aditya', 58000, 'Software Engineer', 4);
44
45 DELIMITER //
46
47 CREATE TRIGGER SalaryValidation
48 BEFORE INSERT ON Employee
49 FOR EACH ROW
50 BEGIN
51     IF NEW.Salary <= 0 THEN
52         SIGNAL SQLSTATE '45000'
53         SET MESSAGE_TEXT = 'Salary must be greater than zero';
54     END IF;
55 END//
56
57 CREATE TRIGGER SalaryValidationUpdate
58 BEFORE UPDATE ON Employee
59 FOR EACH ROW
```

```

60 BEGIN
61   IF NEW.Salary <= 0 THEN
62     SIGNAL SQLSTATE '45000'
63     SET MESSAGE_TEXT = 'Salary must be greater than zero';
64   END IF;
65 END//
66
67 CREATE TRIGGER EmployeeAuditLog
68 AFTER UPDATE ON Employee
69 FOR EACH ROW
70 BEGIN
71   IF OLD.Salary <> NEW.Salary OR OLD.DeptID <> NEW.DeptID THEN
72     INSERT INTO EmployeeAudit
73     (EmpID, OldSalary, NewSalary, OldDeptID, NewDeptID, ActionType)
74     VALUES
75     (OLD.EmpID, OLD.Salary, NEW.Salary, OLD.DeptID, NEW.DeptID, 'EMPLOYEE UPDATED');
76   END IF;
77 END//
78
79 CREATE PROCEDURE transfer_employee(
80   IN emp_id INT,
81   IN new_dept_id INT
82 )
83 BEGIN
84   DECLARE employee_count INT DEFAULT 0;
85   DECLARE department_count INT DEFAULT 0;
86   DECLARE old_dept_id INT;
87
88   DECLARE EXIT HANDLER FOR SQLEXCEPTION
89   BEGIN
90     ROLLBACK;

```

```

83 BEGIN
84   BEGIN
85     RESIGNAL;
86   END;
87
88   START TRANSACTION;
89
90   SELECT COUNT(*)
91   INTO employee_count
92   FROM Employee
93   WHERE EmpID = emp_id;
94
95   IF employee_count = 0 THEN
96     SIGNAL SQLSTATE '45000'
97     SET MESSAGE_TEXT = 'Employee does not exist';
98   END IF;
99
100   SELECT COUNT(*)
101   INTO department_count
102   FROM Department
103   WHERE DeptID = new_dept_id;
104
105   IF department_count = 0 THEN
106     SIGNAL SQLSTATE '45000'
107     SET MESSAGE_TEXT = 'Department does not exist';
108   END IF;
109
110   SELECT DeptID
111   INTO old_dept_id
112   FROM Employee
113   WHERE EmpID = emp_id;

```

```

119 WHERE EmpID = emp_id;
120
121 IF old_dept_id = new_dept_id THEN
122   SIGNAL SQLSTATE '45000'
123   SET MESSAGE_TEXT = 'Employee is already in this department';
124 END IF;
125
126 UPDATE Employee
127 SET DeptID = new_dept_id
128 WHERE EmpID = emp_id;
129
130 COMMIT;
131 END//
132
133 DELIMITER ;
134
135 CALL transfer_employee(101, 2);
136
137 SELECT *
138 FROM Employee
139 WHERE EmpID = 101;
140
141 -- Error-case tests are omitted here because SIGNAL errors stop the script.
142
143 UPDATE Employee
144 SET Salary = 70000
145 WHERE EmpID = 101;
146
147 SELECT *
148 FROM EmployeeAudit;

```

```

83 BEGIN
125
126 UPDATE Employee
127 SET DeptID = new_dept_id
128 WHERE EmpID = emp_id;
129
130 COMMIT;
131 END//
132
133 DELIMITER ;
134
135 CALL transfer_employee(101, 2);
136
137 SELECT *
138 FROM Employee
139 WHERE EmpID = 101;
140
141 -- Error-case tests are omitted here because SIGNAL errors stop the script.
142
143 UPDATE Employee
144 SET Salary = 70000
145 WHERE EmpID = 101;
146
147 SELECT *
148 FROM EmployeeAudit;
149
150 CALL transfer_employee(101, 3);
151
152 SELECT *
153 FROM EmployeeAudit
154 ORDER BY AuditID DESC;

```

OUTPUT







RIDDHIMA VERMA

I/O

AI Agent

78 ms

STDIN

Input for the program (Optional)

Output:

EmpID	EmpName	Salary	JobTitle	DeptID
101	Aarav	55000.00	Software Engineer	2

AuditID	EmpID	OldSalary	NewSalary	OldDeptID	NewDeptID	ActionTime	ActionType
1	101	55000.00	55000.00	1	2	2026-09-26 21:00:10	EMPLOYEE UPDATED
2	101	55000.00	70000.00	2	2	2026-09-26 21:00:10	EMPLOYEE UPDATED

AuditID	EmpID	OldSalary	NewSalary	OldDeptID	NewDeptID	ActionTime	ActionType
3	101	70000.00	70000.00	2	3	2026-09-26 21:00:10	EMPLOYEE UPDATED
2	101	55000.00	70000.00	2	2	2026-09-26 21:00:10	EMPLOYEE UPDATED
1	101	55000.00	55000.00	1	2	2026-09-26 21:00:10	EMPLOYEE UPDATED

OUTCOME/OBSERVATION

- 1.The transfer_employee stored procedure was successfully created with input validation and error handling.
- 2.Valid employee transfers between departments were successfully performed.
- 3.Invalid employee and department IDs were rejected by the procedure.
- 4.Transfers to the employee's existing department were prevented.
- 5.The salary validation triggers successfully rejected zero, negative, and NULL salary values.
- 6.Valid salary modifications were successfully allowed.
- 7.The salary audit trigger automatically recorded salary changes in the SalaryAudit table.
- 8.Edge cases were tested to verify the reliability of the procedure and triggers.